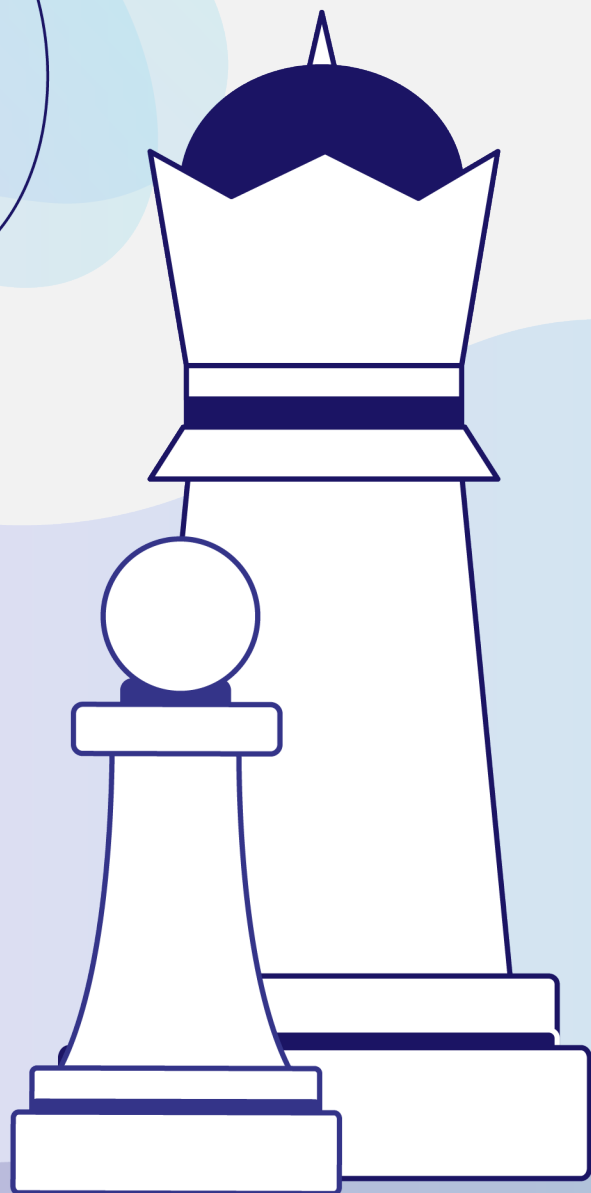




Identificación de poblaciones



En este manual de ejercicios usarás lo aprendido en **Manipulación de Bases de Datos No-SQL: MongoDB** para resolver cuatro misiones que pondrán a prueba tus conocimientos y habilidades adquiridas.

Responde cada actividad en este archivo interactivo, siguiendo las instrucciones correspondientes. Una vez que hayas terminado, dirígete al final del manual para consultar las respuestas correctas y compáralas con las tuyas. Si lo consideras necesario, puedes regresar a las actividades para aprovechar la retroalimentación.



¿Recuerdas la historia de la Real Academia de Ciencias de Suecia?

A lo largo de más de un siglo, el Premio Nobel ha sido uno de los reconocimientos más prestigiosos del mundo. Sin embargo, la Real Academia de Ciencias de Suecia ha notado que, a lo largo de los años, el premio ha sido otorgado a un segmento limitado de la población. Con el objetivo de fomentar una mayor diversidad y asegurar que más personas que contribuyen al avance de la física, la química, la economía o la literatura sean reconocidas, la Fundación Nobel ha decidido emprender un análisis exhaustivo de los datos históricos del premio.

Planteamiento de la situación

La Fundación Nobel ha decidido emprender un análisis detallado. Su objetivo es detectar cuáles segmentos de la población han tenido menor representación en la historia del premio. Para ello, se plantea analizar características como el **género**, la **localidad** y el **país de origen** de los premiados. Se espera que este estudio sirva como base para desarrollar políticas más inclusivas, y que inspire a nuevas generaciones a verse reflejadas en quienes han sido reconocidos por sus aportes a la humanidad.

La base de datos histórica de la Fundación está disponible en formato JSON, pero presenta estructuras no uniformes entre registros. Por esta razón, se ha optado por utilizar **MongoDB**, una base de datos NoSQL ideal para almacenar y manipular grandes volúmenes de datos semiestructurados.

Para ello, convocó a un equipo de expertos en ciencia de datos, entre los cuales se encuentra Marcela, quien propuso utilizar una base de datos NoSQL para facilitar la organización, exploración y análisis de grandes volúmenes de datos con estructuras variables. La información se encuentra en formato JSON y está disponible públicamente en la plataforma Kaggle.

En esta práctica, asumirás el rol de quien continúa el trabajo de Marcela, enfrentándote al reto de importar, explorar, consultar y transformar estos datos utilizando MongoDB y Python. A lo largo de cuatro misiones prácticas, desarrollarás habilidades clave para manipular información en este sistema de base de datos NoSQL, con el objetivo de identificar patrones en la entrega del Premio Nobel y promover una representación más justa e incluyente.

Autoevaluación

Antes de comenzar, es momento de que reflexiones sobre las herramientas y habilidades prácticas que pueden emplearse para gestionar, consultar y automatizar datos utilizando MongoDB, reconociendo el valor de las bases de datos NoSQL en la resolución de problemas reales.

Instrucciones: Lee con atención las siguientes frases y selecciona las que mejor describen cómo te percibes:

Entiendo la diferencia entre un sistema de base de datos SQL y uno NoSQL.	
Comprendo las circunstancias en las que tiene ventajas usar una base de datos NoSQL.	
Reconozco la manera en que los datos se almacenan en una base MongoDB.	
Puedo realizar consultas sobre los datos contenidos en una colección de MongoDB.	
Puedo realizar agregaciones sobre colecciones de MongoDB.	
Puedo acceder a MongoDB y hacer consultas y agregaciones desde Python.	

Si marcaste todos los elementos de la lista, **¡felicidades!** Estás listo para practicar estos conocimientos a través de un caso. Si dejaste algún elemento sin marcar, te invitamos a que revises de nuevo el tema correspondiente y después retomes esta actividad.



Introducción a las misiones



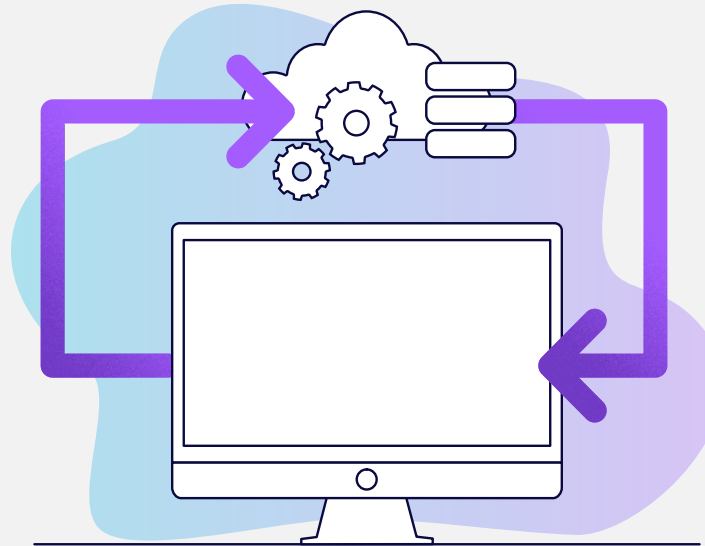
Ahora que comprobaste que cuentas con los conocimientos necesarios, es momento de que apliques todo lo aprendido en una serie de ejercicios que tenemos preparados para ti.

Para completar esta práctica, deberás enfrentarte y superar **cuatro misiones**, cada una de ellas muestra entre dos y tres ejercicios en los que aplicarás lo revisado en las lecciones del curso.

¿Estás listo? ¡Comencemos con la primera misión!

Misión 1: Preparando tu entorno y tus datos

Propósito: Comprender los conceptos fundamentales de las bases de datos NoSQL, conocer las características principales de MongoDB y aprender a importar datos en un entorno de trabajo en la nube.



Situación: Como parte del análisis del Premio Nobel, necesitas preparar un entorno de trabajo que te permita almacenar y consultar grandes volúmenes de información. Dado que los archivos disponibles tienen estructura JSON y presentan ligeras variaciones, MongoDB es la alternativa ideal.

Misión 1: Ejercicio 1

Introducción: Para comenzar a trabajar con los datos, primero necesitas conectarte al servidor. Para esto, requieres una cadena de conexión.

Instrucciones: Selecciona la opción que representa correctamente una cadena de conexión válida para conectarte a MongoDB Atlas.

Opciones de respuesta		
a)	<code>mongodb+srv://<db_username>:<db_password>@cluster0.otxeh4d.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0</code>	
b)	<code>mongodb+srv://cluster0.otxeh4d.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0</code>	
c)	<code>mongodb+srv://<db_username>:<db_password>@localhost/?retryWrites=true&w=majority&appName=Cluster0</code>	

Revisar respuestas >

Misión 1: Ejercicio 2

Introducción: Para conectarse a MongoDB, Python emplea una biblioteca llamada PyMongo.

Instrucciones: Lee con atención el siguiente planteamiento y elige la respuesta correcta.

¿Cuál sería el código de Python para conectarse al servidor de MongoDB Atlas?		
a)	<pre>from pymongo import MongoClient client = newConnection("localhost")</pre>	
b)	<pre>from pymongo import MongoClient client = MongoClient.connect("localhost")</pre>	
c)	<pre>from pymongo import MongoClient client = MongoClient(<cadena de conexión>)</pre>	

Revisar respuestas >

Misión 1: Ejercicio 3

Instrucciones: Lee con atención el siguiente planteamiento y elige la respuesta correcta.

A partir de la conexión creada en el ejercicio anterior, misma que está almacenada en la variable `client`, ¿cuál sería la manera correcta de referenciar la base de datos `nobel` y, a su vez, referenciar a la colección `laureates`? Es necesario asignar cada referencia a una variable.

Opciones de respuesta		
a)	<pre>nobel = client.db(nobel) laureates = nobel.collection(laureates)</pre>	
b)	<pre>nobel = client.nobel laureates = nobel.laureates</pre>	
c)	<pre>nobel = client.get('nobel') laureates = nobel.get('laureates')</pre>	

Revisar respuestas >

¡Felicidades! En esta primera misión comprendiste los fundamentos de MongoDB como base de datos NoSQL. Este paso es fundamental, ya que te permite trabajar con estructuras no uniformes y aprovechar las ventajas de una base de datos NoSQL para consultas flexibles y escalables.

¡Vas en camino! Terminaste tu primera misión; aún tienes tres más por resolver. Continúa con el material para completar cada una de ellas.



Misión 2: Consultando y explorando la información

Propósito: Consultar la información almacenada en la colección `laureates` mediante comandos básicos de MongoDB, con el fin de obtener datos específicos sobre los ganadores del Premio Nobel a lo largo de la historia.

Situación: Ya tienes cargada la base de datos con los registros de quienes han sido galardonados con el Premio Nobel. Ahora necesitas comenzar a explorar esa información: ¿cuántas mujeres lo han recibido?, ¿qué países tienen más premios?, ¿qué personas han sido reconocidas más de una vez?

En esta misión practicarás cómo realizar consultas simples para encontrar información clave que pueda ayudar a construir un análisis más justo e inclusivo.

Misión 2: Ejercicio 1

Introducción: Para hacer consultas de MongoDB a partir de las referencias de bases de datos y colecciones, PyMongo toma un diccionario de Python y automáticamente lo convierte en un documento de MongoDB. Las consultas regresan un objeto de tipo **Cursor**, mismo que puede iterarse o convertirse en una lista.

Instrucciones: Lee con atención el siguiente planteamiento y elige la respuesta correcta.

Considerando la referencia a la colección `laureates` definida en la misión anterior, ¿cuál de las siguientes instrucciones ejecutará consultas y almacenará el resultado en **listas de Python** en dos variables diferentes para hombres y mujeres que han ganado un premio?

Opciones de respuesta		
a)	<pre>laureates_female = list(laureates.find({"gender":"female"})) laureates_male = list(laureates.find({"gender":"male"}))</pre>	
b)	<pre>laureates_female = laureates.find({"gender":"female"}) laureates_male = laureates.find({"gender":"male"})</pre>	
c)	<pre>laureates_female = laureates.find({"gender":"female"}).toList() laureates_male = laureates.find({"gender":"male"}).toList()</pre>	

Revisar respuestas >

Misión 2: Ejercicio 2

Introducción: Uno de los objetivos del análisis es identificar patrones de representación de género en los premios otorgados. Comenzarás revisando cuántas personas han sido identificadas como mujeres en la base de datos.

Instrucciones: Completa el siguiente comando para contar cuántas mujeres han recibido el Premio Nobel, según la base de datos:

```
nobel.laureates.count_documents ( { " _____ " : " _____ " } )
```

[Revisar respuestas >](#)



Misión 2: Ejercicio 3

Introducción: Saber cuántos premios Nobel se han otorgado a personas de cada país es clave para identificar regiones subrepresentadas.

Instrucciones: Completa el siguiente comando para contar cuántos laureados tienen como país de nacimiento "Mexico".

```
nobel.laureates.count_documents({ _____: _____ })
```

[Revisar respuestas >](#)



¡Bien hecho! En esta misión pusiste en práctica consultas simples para explorar los datos de la colección `laureates`. Repasaste cómo recuperar documentos, aplicar filtros y realizar conteos específicos, herramientas fundamentales para iniciar cualquier análisis de datos con MongoDB. Estás un paso más cerca de generar hallazgos útiles para construir un panorama más justo del Premio Nobel.

¡Vas en camino! Terminaste tu segunda misión; aún tienes dos más por resolver. Continúa con el material para completar cada una de ellas.



Misión 3: Profundizando con agregaciones

Propósito: Utilizar el *framework* de agregación de MongoDB para realizar consultas más complejas que permitan analizar tendencias y patrones dentro de la base de datos del Premio Nobel.

Situación: Hasta ahora, has realizado consultas básicas para conocer la estructura de los datos y explorar campos clave como el país o el género de los laureados. Sin embargo, para responder preguntas más complejas, necesitas aplicar operaciones de agregación.

MongoDB permite agrupar, filtrar y transformar documentos con el uso de *pipelines* de agregación. En esta misión, trabajarás con operadores como `$match`, `$group` y `$project` para realizar análisis más profundos.

Misión 3: Ejercicio 1

Introducción: Un *pipeline* de agregaciones es una herramienta de MongoDB que permite encadenar comandos de agregación para manipular u obtener resultados más complejos que con consultas simples. Recuerda que las agregaciones permiten hacer agrupaciones sobre campos específicos, modificar o crear nuevos campos o filtrar por condiciones predeterminadas.

Instrucciones: Lee con atención el siguiente planteamiento y elige la respuesta correcta.

¿Cuál de las siguientes opciones regresa un resultado con el total de premios recibidos por hombres y mujeres y lo almacena en una variable llamada `laureates_by_gender`?

Opciones de respuesta		
a)	<pre>pipeline_gender = "SELECT gender, count(*) FROM laureates GROUP BY gender" laureates_by_gender = list(laureates.aggregate(pipeline_gender))</pre>	
b)	<pre>pipeline_gender = [{ '\$match': { 'gender': { '\$exists': 1 } } }, { '\$group': { '_id': '\$gender',</pre>	

	<pre> 'total_laureates': { '\$sum': 1 } }] laureates_by_gender = list(laureates.aggregate(pipeline_gender)) </pre>	
c)	<pre> pipeline_gender = [{ '\$match': { 'gender': { '\$exists': 1 } } }, { '\$group': { 'field': '\$gender', 'total_laureates': { '\$sum': 1 } } }] laureates_by_gender = list(laureates.aggregate(pipeline_gender)) </pre>	

[Revisar respuestas >](#)

Misión 3: Ejercicio 2

Introducción: En algunos casos, no necesitas todos los campos del documento, sino solo algunos específicos. MongoDB permite proyectar aquellos que te interesa mostrar. Por ejemplo, puedes decidir mostrar solo el nombre completo de cada laureado y su país de nacimiento.

Instrucciones: Completa el siguiente *pipeline* para mostrar solo los campos de nombre y país, ocultando `_id`.

```
pipeline = [  
    { "$_____": { "_id": _____, "knowName.es": _____, "birth.place.country.en": _____ } }  
]
```

[Revisar respuestas >](#)



¡Buen trabajo! En esta misión aprendiste a aplicar el *framework* de agregación de MongoDB para analizar los datos de forma más poderosa. Usaste `$match` para filtrar, `$group` para agrupar y contar, y `$project` para mostrar solo los datos relevantes. Estas habilidades te permiten transformar conjuntos grandes y complejos de datos en respuestas claras para preguntas concretas.

¡Estás a punto de terminar, solo te falta una misión! Continúa con el material para completarla.



Misión 4: Automatizando el análisis con Python y MongoDB

Propósito: Establecer una conexión con MongoDB desde Python utilizando la librería `pymongo`, consultar y manipular documentos en la base de datos de forma programática.

Situación: Después de explorar tus datos directamente desde MongoDB, te das cuenta de que hacer este tipo de consultas manualmente puede volverse repetitivo y limitar tus posibilidades de análisis. Para automatizar tareas, procesar grandes volúmenes de información y crear reportes dinámicos, es mejor integrar tu base de datos con Python.

En esta misión establecerás una conexión desde Python utilizando `pymongo`, y comenzarás a hacer consultas y manipulaciones directamente desde código. Esto te dará mayor flexibilidad para trabajar con los datos históricos del Premio Nobel.

Misión 4: Ejercicio 1

Introducción: Ya tienes MongoDB corriendo en Colab. Ahora necesitas establecer una conexión desde Python para poder consultar y manipular los datos programáticamente.

Instrucciones: Selecciona la opción que representa la forma adecuada de conectarte a un servidor local de MongoDB usando pymongo.

a)	<code>connect = MongoClient("localhost")</code>	☐
b)	<code>import MongoClient</code>	☐
c)	<code>from pymongo import MongoClient; client = MongoClient()</code>	☐
d)	<code>MongoClient(pymongo)</code>	☐

Revisar respuestas >

Misión 4: Ejercicio 2

Introducción: Estás trabajando desde Python para explorar los datos del Premio Nobel y necesitas consultar a todos los galardonados que han ganado más de una vez. En el documento JSON, la clave `nobelPrizes` contiene un arreglo con todos los premios que ha recibido una persona. Si ese arreglo tiene más de un elemento, significa que fue galardonada múltiples veces.

Instrucciones: Elige la instrucción correcta en `pymongo` que filtra documentos donde el arreglo `nobelPrizes` tiene más de un elemento.

a)	<code>nobel.laureates.find({ "nobelPrizes": { "\$length": { "\$eq": 2 } } })</code>	
b)	<code>nobel.laureates.find({ "nobelPrizes.length": { "\$gt": 1 } })</code>	
c)	<code>nobel.laureates.find({ "nobelPrizes": { "\$size": { "\$gt": 1 } } })</code>	
d)	<code>nobel.laureates.find({ "nobelPrizes": { "\$size": 2 } })</code>	

[Revisar respuestas >](#)

¡Buen trabajo! Con esta misión diste el paso hacia la automatización del análisis de datos al conectar MongoDB con Python. Aprendiste a consultar, filtrar y manipular documentos desde código, lo cual te permitirá integrar estas habilidades en proyectos reales y realizar análisis a mayor escala, de forma más eficiente y reproducible.



Cierre

Muchos de los problemas comunes relacionados con el análisis de datos pueden resolverse eficientemente si sabes cómo organizar, consultar y transformar la información contenida en una base de datos. Cuando los datos tienen estructuras flexibles y no uniformes, como en este caso con documentos JSON, **MongoDB** se convierte en una herramienta poderosa que permite acceder a la información de forma escalable y dinámica.

Además, al integrar MongoDB con **Python**, amplias tus capacidades como analista o desarrollador, ya que puedes automatizar procesos, generar reportes personalizados y aplicar herramientas estadísticas o gráficas para mejorar la toma de decisiones basada en datos.

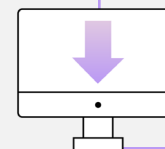
Las habilidades que desarrollaste en esta práctica te dan una base sólida para elegir la tecnología de base de datos más adecuada para tus proyectos, y para sacar el máximo provecho de las ventajas de MongoDB cuando se trata de manejar datos con estructuras variadas. Completaste con éxito cuatro misiones clave en las que:

- Consultaste y exploraste documentos en MongoDB, aplicando filtros por criterios como género y país de nacimiento, y comprendiendo cómo acceder a la información relevante para identificar patrones.
- Aplicaste *pipelines* de agregación usando operadores como `$match`, `$group` y `$project` para realizar análisis más complejos y descubrir tendencias en la entrega de los premios.
- Automatizaste el análisis desde Python al conectar MongoDB con `pymongo`, ejecutando consultas desde código, clasificando funciones según su propósito, y utilizando filtros sobre campos específicos y arreglos para identificar casos especiales, como los laureados con múltiples premios.



Ahora que conoces el ciclo completo —desde importar y consultar información, hasta automatizar tareas con código—, es momento de aplicar lo aprendido en un **reto final**.

En esta actividad integradora, deberás utilizar los recursos que ya dominaste para responder nuevas preguntas relacionadas con los Premios Nobel. Será tu oportunidad para demostrar que puedes trabajar de forma autónoma con MongoDB y Python, y entregar evidencia clara de tu aprendizaje.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.

Misión 1 / Ejercicio 1

Solución:		
a)	<code>mongodb+srv://<db_username>:<db_password>@cluster0.otxeh4d.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0</code>	☒
b)	<code>mongodb+srv://cluster0.otxeh4d.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0</code>	☐
c)	<code>mongodb+srv://<db_username>:<db_password>@localhost/?retryWrites=true&w=majority&appName=Cluster0</code>	☐

Retroalimentación: La respuesta correcta es la A. Si no fue la respuesta que elegiste, consulta nuevamente el material. Recuerda que una cadena válida para MongoDB Atlas debe incluir el nombre de usuario y contraseña (solo asegúrate de reemplazar `<db_username>` y `<db_password>` por tus credenciales reales), así como el clúster correcto. Además, no utilices direcciones `localhost` en servicios en la nube.

[< Regresar a la misión](#)

Misión 1 / Ejercicio 2

Solución:		
a)	<pre>from pymongo import MongoClient client = newConnection("localhost")</pre>	☐
b)	<pre>from pymongo import MongoClient client = MongoClient.connect("localhost")</pre>	☐
c)	<pre>from pymongo import MongoClient client = MongoClient(<cadena de conexión>)</pre>	☒

Retroalimentación: La respuesta correcta es la C. Los comandos de las opciones A y B solamente son válidos para servidores locales. Recuerda que el servidor se indica por la <cadena de conexión>, y en la variable `client` se almacena una conexión válida a la base de datos.

[< Regresar a la misión](#)

Misión 1 / Ejercicio 3

Solución:		
a)	<pre>nobel = client.db(nobel) laureates = nobel.collection(laureates)</pre>	☐
b)	<pre>nobel = client.nobel laureates = nobel.laureates</pre>	☒
c)	<pre>nobel = client.get('nobel') laureates = nobel.get('laureates')</pre>	☐

Retroalimentación: La respuesta correcta es la B. Si no fue la respuesta que elegiste, consulta nuevamente el material. Recuerda que a las bases de datos y a las colecciones se puede acceder directamente como propiedades del objeto de Python.

[< Regresar a la misión](#)

Misión 2 / Ejercicio 1

Solución:		
a)	<pre>laureates_female = list(laureates.find({"gender":"female"})) laureates_male = list(laureates.find({"gender":"male"}))</pre>	☒
b)	<pre>laureates_female = laureates.find({"gender":"female"}) laureates_male = laureates.find({"gender":"male"})</pre>	☐
c)	<pre>laureates_female = laureates.find({"gender":"female"}).toList() laureates_male = laureates.find({"gender":"male"}).toList()</pre>	☐

Retroalimentación: La respuesta correcta es la A. La opción B solamente ejecuta la consulta y guarda el Cursor en la variable, pero no la convierte en lista. Por otro lado, la opción C es incorrecta, ya que no existe el método `toList` en el objeto Cursor.

[< Regresar a la misión](#)

Misión 2 / Ejercicio 2

Solución:

```
nobel.laureates.count_documents({ "gender": "female" })
```

Retroalimentación: Si acertaste, acabas de filtrar por género femenino correctamente. Este tipo de consultas te permite identificar patrones clave sobre representación. Si no fue así, revisa la estructura del filtro. Recuerda que MongoDB usa sintaxis JSON con llaves para definir los criterios de búsqueda.

[< Regresar a la misión](#)

Misión 2 / Ejercicio 3

Solución:

```
nobel.laureates.count_documents({ "birth.place.countryNow.en": "Mexico" })
```

Retroalimentación: Si acertaste, has contado con éxito cuántos premios Nobel han sido otorgados a personas nacidas en México. Esta consulta te será útil para comparar representaciones por país. Si no fue así, verifica que estás usando el campo correcto (`birth.place.countryNow.en`) y que el valor está escrito tal como aparece en los documentos: "Mexico". No olvides usar las llaves para definir el criterio.

[< Regresar a la misión](#)

Misión 3 / Ejercicio 1

Solución:		
a)	<pre>pipeline_gender = "SELECT gender, count(*) FROM laureates GROUP BY gender" laureates_by_gender = list(laureates.aggregate(pipeline_gender))</pre>	
b)	<pre>pipeline_gender = [{ '\$match': { 'gender': { '\$exists': 1 } } }, { '\$group': { '_id': '\$gender', 'total_laureates': { '\$sum': 1 } } }] laureates_by_gender = list(laureates.aggregate(pipeline_gender))</pre>	

c)

```
pipeline_gender = [  
    {  
        '$match': {  
            'gender': {  
                '$exists': 1  
            }  
        }  
    }, {  
        '$group': {  
            'field': '$gender',  
            'total_laureates': {  
                '$sum': 1  
            }  
        }  
    }  
]  
  
laureates_by_gender = list(laureates.aggregate(pipeline_gender))
```

Retroalimentación: La respuesta correcta es la B, ya que la función debe ser escrita exactamente de esta forma. La opción A es inválida porque las consultas de SQL no son compatibles con MongoDB. Por último, la opción C es incorrecta, ya que la sintaxis de la etapa `$group` no es adecuada.

[< Regresar a la misión](#)

Misión 3 / Ejercicio 2

Solución:

```
pipeline = [  
  {"$project": {"_id": 0, "knowName.es": 1, "birth.place.country.en": 1 }}]
```

Retroalimentación: ¡Muy bien! Recuerda que para ocultar un campo, debes asignarle el valor "0", y para mostrarlo, "1". En MongoDB, el campo `_id` se muestra por defecto, por eso debes ocultarlo explícitamente si no lo necesitas.

[< Regresar a la misión](#)

Misión 4 / Ejercicio 1

Solución:		
a)	<code>connect = MongoClient("localhost")</code>	☐
b)	<code>import MongoClient</code>	☐
c)	<code>from pymongo import MongoClient; client = MongoClient()</code>	☒
d)	<code>MongoClient(pymongo)</code>	☐

Retroalimentación: La respuesta correcta es la C. Si no fue la respuesta que elegiste, consulta nuevamente el material. `MongoClient()` es el método adecuado para conectarse a un servidor local usando `pymongo`. Recuerda que el cliente se importa desde `pymongo` y se instancia sin parámetros en entornos locales.

[< Regresar a la misión](#)

Misión 4 / Ejercicio 2

Solución:		
a)	<code>nobel.laureates.find({ "nobelPrizes": { "\$length": { "\$eq": 2 } } })</code>	☐
b)	<code>nobel.laureates.find({ "nobelPrizes.length": { "\$gt": 1 } })</code>	☐
c)	<code>nobel.laureates.find({ "nobelPrizes": { "\$size": { "\$gt": 1 } } })</code>	☐
d)	<code>nobel.laureates.find({ "nobelPrizes": { "\$size": 2 } })</code>	☒

Retroalimentación: La respuesta correcta es la D. Si no fue la respuesta que elegiste, consulta nuevamente el material. Recuerda que operadores como `$size` no permiten directamente comparaciones como `$gt`, por lo que si queremos obtener la lista de ganadores con más de un premio (2 o más), es necesario realizar una agregación.

[< Regresar a la misión](#)